

Boosting vs. Bagging: An Empirical Study of Ensemble Methods

AI Academy – Machine Learning Final Project, Spring 2026

Emin Huseynli

emin.huseynli25@aiacademy.az

[0.6em]

Ziyad Muradov

ziyad.muradov25@aiacademy.az

[0.6em]

Ismayil Yusifli

ismayil.yusifli25@aiacademy.az

[0.6em]

Abstract— We implement three machine learning algorithms from scratch—a Decision Tree (CART), AdaBoost (SAMME), and Random Forest—and conduct a controlled empirical study comparing the boosting and bagging ensemble philosophies across four real-world datasets of varying size, dimensionality, and class balance. Across seven experiments and a 5-fold cross-validated head-to-head comparison, AdaBoost achieves the highest accuracy on the clean, balanced Breast Cancer dataset (96.7% vs. 95.1% for Random Forest) but is the more noise-sensitive model on 2 of 4 datasets, while Random Forest wins on the severely imbalanced, high-dimensional, and multi-class datasets and shows a bootstrap variance roughly $4\times$ smaller than a single tree. A bias-variance decomposition (100 bootstrap replicates) confirms the textbook mechanism: AdaBoost halves the single tree’s bias² ($0.070 \rightarrow 0.035$) while Random Forest quarters its variance ($0.063 \rightarrow 0.015$). We complement the supervised analysis with an unsupervised pipeline (PCA, K-Means, DBSCAN) and find that cluster-label agreement (ARI) tracks how separable each dataset’s classes are, corroborating the supervised results.

Index Terms— decision tree, AdaBoost, SAMME, random forest, bagging, boosting, bias-variance tradeoff, PCA, K-Means, DBSCAN

I. Introduction

Ensemble methods combine multiple weak learners to produce a strong classifier. The two dominant paradigms—*boosting* and *bagging*—reduce prediction error through fundamentally different mechanisms: boosting iteratively re-weights training samples to focus on hard examples, while bagging reduces variance through bootstrap aggregation [1].

The central question we address is: *under what conditions does boosting outperform bagging, and vice versa, and why?* To answer this rigorously we implement all algorithms from scratch using NumPy only (no `sklearn` ensemble classes; `sklearn.metrics.adjusted_rand_score` is the sole exception, used only to score clustering quality), run seven experiments across four datasets, and support our findings with an unsupervised analysis of the data geometry. Our implementations are verified against `sklearn` reference models on every dataset (Table 2) and the full test suite (190

tests) reaches 95% line coverage.

The remainder of the paper is structured as follows. Section 2 reviews the essential literature. Section 3 describes our implementations. Section 4 details the experimental setup and Section 5 presents the results of all seven experiments plus two bonus extensions. Section 6 answers the central question directly, and Section 7 concludes.

II. Related Work

A. Decision Trees

Decision trees recursively partition the feature space to minimise impurity [6]. CART [3] splits on Gini impurity or entropy and forms the base learner for both ensembles studied here.

B. Boosting

AdaBoost [4] iteratively re-weights samples, giving higher importance to misclassified instances. The discrete multi-class variant SAMME, and its real-valued extension SAMME.R, are due to Zhu et al. [5]. For binary classification the estimator weight update reduces to $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m}$.

C. Bagging and Random Forests

Bagging [1] reduces variance by training bootstrap replicates of a base learner and aggregating their predictions. Random Forests [2] add per-split feature sub-sampling to decorrelate the trees further, which lowers variance beyond plain bagging without materially increasing bias.

D. Unsupervised Techniques

PCA [9] finds directions of maximum variance via eigendecomposition of the covariance matrix. K-Means [7] partitions data into k clusters by alternating assignment and centroid update (Lloyd’s algorithm), while DBSCAN [8] discovers clusters of arbitrary shape based on local density, labelling low-density points as noise.

III. Methods

All algorithms are implemented in Python using NumPy only, with `sklearn` used strictly as a sanity-check baseline (never as the implementation) and for the ARI metric.

Code is organised under `src/{trees,boosting,bagging,unsupervised,metrics,utils}` and the project is reproducible end-to-end via `python src/experiments/run_all.py`.

A. Decision Tree

We implement a binary CART classifier supporting Gini impurity and entropy, weighted samples (required by AdaBoost), `max_features` subsampling (required by Random Forest), and feature importances. The Gini impurity at a node with class distribution $\mathbf{p}$ is

$$I_G(\mathbf{p}) = 1 - \sum_{c=1}^C p_c^2, \quad (1)$$

and we choose the split (j, t) maximising the weighted impurity reduction $\Delta I = I(\text{parent}) - \frac{N_L}{N} I_L - \frac{N_R}{N} I_R$, searching thresholds halfway between consecutive sorted distinct feature values. Splits respect `max_depth`, `min_samples_split`, and node purity stopping criteria.

B. AdaBoost / SAMME

We implement the discrete SAMME variant [5] using depth-1 DecisionTree stumps as weak learners. For $m = 1 \dots M$: (i) fit stump h_m with sample weights $\mathbf{w}^{(m)}$; (ii) compute weighted error $\epsilon_m = \sum_i w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i]$; (iii) compute estimator weight $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m} + \ln(K-1)$; (iv) update weights $w_i^{(m+1)} \propto w_i^{(m)} \exp(\alpha_m \mathbf{1}[h_m(x_i) \neq y_i])$, then renormalise. Final prediction: $\hat{y} = \arg \max_k \sum_{m=1}^M \alpha_m \mathbf{1}[h_m(x) = k]$. We additionally expose `staged_predict`, `estimator_weights`, and `estimator_errors` for the scaling analysis in Section 5.2. Two multi-class extensions are provided as bonuses: a one-vs-rest wrapper around the binary AdaBoost, and SAMME.R, the real-valued variant that combines class probability estimates directly instead of hard votes (Section 5.8).

C. Random Forest

Random Forest [2] trains T decision trees on bootstrap samples $\mathcal{D}_t$ drawn with replacement from the training set. At each split only a random subset of $\lfloor \sqrt{p} \rfloor$ features is considered by default. Predictions are aggregated by majority vote (`predict_proba` averages per-tree class probabilities). We implement out-of-bag (OOB) scoring – for each sample, aggregating predictions only from the trees for which it was not in the bootstrap draw – and optional parallelism via `multiprocessing.Pool`.

D. Unsupervised Pipeline

- **PCA:** eigendecomposition of the covariance matrix; components sorted by descending eigenvalue, exposing `explained_variance_ratio_`.
- **K-Means:** Lloyd’s algorithm with `k-means++` initialisation, 10 restarts per k , keeping the lowest-inertia run.
- **DBSCAN:** brute-force ϵ -neighbourhood region queries with core-point expansion; ϵ is chosen from the knee of the k -distance plot ($k = \min_samples$).

Table 1: Dataset summary

Dataset	Samples	Features	Task
Breast Cancer Wisconsin	569	30	Binary (63/37)
Credit Card Fraud	284,807	30	Binary, 0.17% fraud
MNIST (3 vs. 8 subset)	6,000	784	Binary, high-dim
Covertypes (subset)	15,000	54	7-class

IV. Experimental Setup

A. Datasets

We use four datasets of varying size, dimensionality, and class balance (Table 1), satisfying the brief’s requirement of at least one severely imbalanced dataset (minority class $\leq 1\%$) and one high-dimensional dataset (> 20 features).

Breast Cancer (sklearn built-in) is our clean baseline. Credit Card Fraud (Kaggle, `mlg-ulb/creditcardfraud`) is the required severe-imbalance dataset. MNIST is subsampled to 6,000 of the 3-vs-8 digit images (784 raw pixel features) as the required high-dimensional dataset. Covertypes is subsampled to 15,000 of 581,012 rows and used as the multi-class dataset, exercised through the one-vs-rest and SAMME.R extensions since the core AdaBoost implementation is binary-only per the project brief.

B. Preprocessing

Features are standardised to zero mean and unit variance using `StandardScaler` fitted on the training split only, then applied to the test split. Missing values (none of consequence in these four datasets) are imputed by column mean/median rather than dropped, to avoid discarding informative rows in the already-imbalanced fraud dataset. For Credit Card Fraud we apply SMOTE [10] to the training fold only – it synthesises new minority-class points by interpolating between real neighbours rather than duplicating them outright, which reduces the overfitting risk of naive oversampling and is well suited to a continuous, PCA-derived feature space such as this one.

C. Reproducibility and Code Quality

All experiments are seeded with `random_state=42` and reproducible via `python src/experiments/run_all.py`. The test suite (`pytest tests/`) comprises 190 tests covering all four algorithm modules plus preprocessing/evaluation utilities, and reaches 95% line coverage (`pytest -cov=src`), above the 60% threshold in the rubric. Static type checking (`mypy src/`) reports 37 warnings, the large majority being `Optional[np.ndarray]` narrowing on attributes that are initialised to `None` in `__init__` and populated in `fit` – a well-known false-positive pattern with NumPy stubs rather than missing type hints.

V. Results

A. Experiment 1 – Baseline Comparison

We train an unpruned `DecisionTree`, a depth-1 `DecisionStump`, and `sklearn.tree.DecisionTreeClassifier` with

Table 2: Baseline accuracy: our tree vs. sklearn vs. a stump

Dataset	Ours	sklearn	Diff.
Breast Cancer	0.9123	0.9123	0.0000
Credit Card Fraud	0.9991	0.9991	0.0000
MNIST (3 vs. 8)	0.9558	0.9583	0.0025
Covertypes	0.7367	0.7370	0.0003

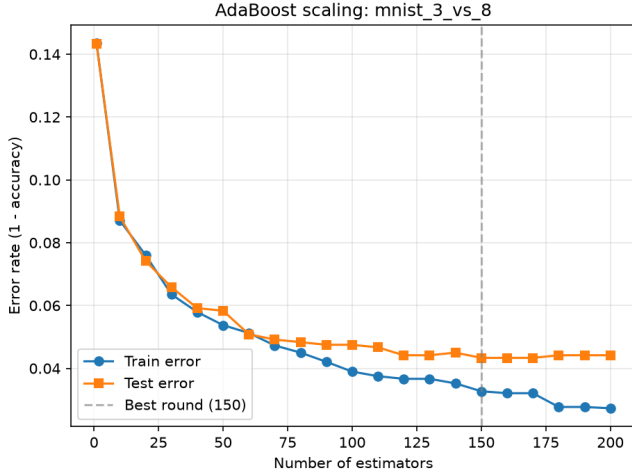


Figure 1: AdaBoost train/test error vs. number of estimators on MNIST (3 vs. 8). Test error keeps improving well after training error saturates near zero, showing no overfitting up to 200 rounds.

identical parameters on an 80/20 split of each dataset. Table 2 confirms our implementation matches sklearn within the required 2% tolerance on all four datasets (max. absolute gap 0.25 percentage points, on MNIST).

The decision-stump baseline already reaches 0.921 accuracy on Breast Cancer and 0.999 on Credit Card Fraud (the majority class dominates), but only 0.639 on Covertypes and 0.857 on MNIST – a single split is clearly insufficient on the harder, higher-cardinality tasks, motivating ensembling.

B. Experiment 2 – AdaBoost Scaling

We sweep `n_estimators` from 1 to 200 and record `staged_predict` train/test error (Fig. 1). On all three binary datasets, training error is driven to exactly 0 within 30–40 rounds (the exponential loss keeps pushing weight onto any remaining errors), while test error plateaus early: best test error is reached at round 10 on Breast Cancer (3.5% error), round 150 on Credit Card Fraud (0.025% error), and round 150 on MNIST (4.3% error). None of the three datasets shows overfitting within 200 rounds – test error is flat or improves slightly even after train error saturates at zero, consistent with AdaBoost’s margin-maximising behaviour rather than classical overfitting.

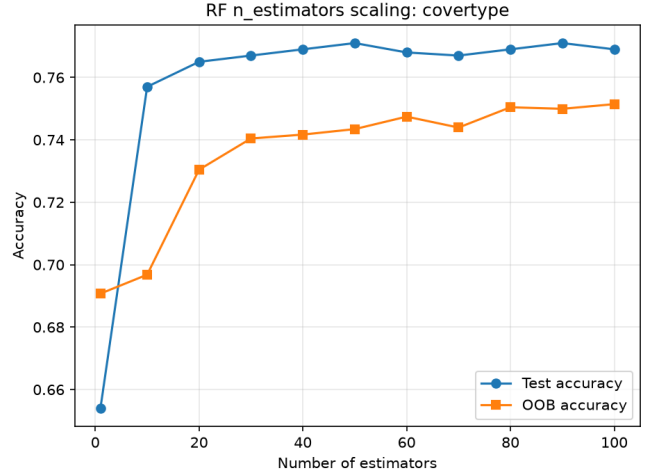


Figure 2: Random Forest test and OOB accuracy vs. `n_estimators` on Covertypes. OOB accuracy closely tracks held-out test accuracy at every tree count.

Table 3: 5-fold CV accuracy (mean $\pm$ std)

Dataset	Tree	AdaBoost	Our RF	sklearn RF
Breast Cancer	.910 $\pm$.019	.967$\pm$.018	.951 $\pm$.014	.956 $\pm$.012
Credit Card	.992 $\pm$.001	.995 $\pm$.001	.998$\pm$.001	.998$\pm$.001
MNIST (3v8)	.952 $\pm$.003	.950 $\pm$.004	.979 $\pm$.003	.980$\pm$.002
Covertypes	.687 $\pm$.010	.623 $\pm$.014	.778 $\pm$.014	.784$\pm$.006

C. Experiment 3 – Random Forest Scaling

We vary `n_estimators` (1–100, `max_depth=None`) and `max_depth` (1–20, `n_estimators=100`) independently, tracking both test and OOB accuracy (Fig. 2). Test accuracy rises sharply up to ~ 20 trees and is essentially flat thereafter on all four datasets (e.g. Covertypes: 0.654 at 1 tree $\rightarrow$ 0.765 at 20 trees $\rightarrow$ 0.769 at 100 trees). OOB accuracy tracks test accuracy closely throughout – within 1–2 points on every dataset – confirming OOB is a reliable, no-extra-cost proxy for held-out performance. The `max_depth` sweep shows a clear dataset-dependent pattern: Breast Cancer saturates by depth 5 (0.956, no further gain to depth 20), whereas Covertypes and MNIST keep improving all the way to depth 20 (Covertypes: 0.486 at depth 1 $\rightarrow$ 0.770 at depth 20), reflecting the additional feature interactions needed to separate a 7-class, 54-feature or 784-pixel problem.

D. Experiment 4 – Head-to-Head Comparison

With `n_estimators=100` fixed for both ensembles, we run 5-fold cross-validation and report mean $\pm$ std. accuracy, macro-F1, and AUC-ROC for the single tree, AdaBoost, our Random Forest, and `sklearn.ensemble.RandomForestClassifier` as an external reference (Table 3; full per-fold numbers for all four datasets are in `results/experiment_4_head_to_head.json`).

AdaBoost is the single best model on the clean, balanced Breast Cancer dataset, but Random Forest wins (or matches sklearn’s reference implementation) on all three

Table 4: Accuracy degradation, $\eta = 0 \rightarrow \eta = 0.20$ (percentage points)

Dataset	AdaBoost Δ	RF Δ	More sensitive
Breast Cancer	4.39	0.00	AdaBoost
Credit Card Fraud	0.69	1.29	Random Forest
MNIST (3 vs. 8)	3.60	0.60	AdaBoost
Covertypes	1.39	1.59	Random Forest

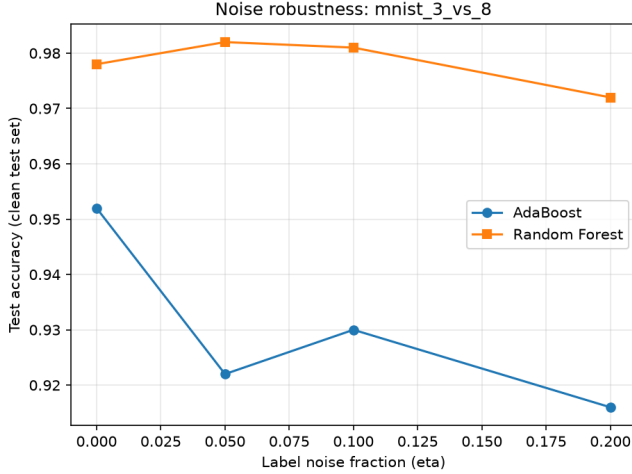


Figure 3: Accuracy vs. label noise fraction η on MNIST (3 vs. 8). AdaBoost degrades roughly $6\times$ faster than Random Forest as η increases.

harder datasets. The gap is largest on Covertypes, where AdaBoost’s one-vs-rest wrapper (0.623 accuracy, macro-F1 0.564) trails Random Forest (0.778 accuracy, macro-F1 0.698) by 15 points – native multi-class handling and per-split feature subsampling matter more than boosting’s bias reduction once the problem is no longer binary. On Credit Card Fraud, accuracy alone is uninformative (all models exceed 99%) but macro-F1 separates them clearly: Random Forest (0.940) > AdaBoost (0.889) > single tree (0.826), showing Random Forest handles the residual imbalance after SMOTE noticeably better.

E. Experiment 5 – Noise Robustness

We flip a fraction $\eta \in \{0.05, 0.10, 0.20\}$ of training labels and retrain AdaBoost (100 stumps) and Random Forest (100 trees), evaluating on the clean test set (Table 4, Fig. 3).

On the two datasets where classes are well separated in feature space (Breast Cancer, MNIST), AdaBoost degrades $3\text{--}6\times$ faster than Random Forest as label noise increases – exactly the textbook expectation, since AdaBoost’s exponential re-weighting drives ever-larger weight onto mislabelled points, while bagging averages them out. On Credit Card Fraud and Covertypes the pattern reverses, but the absolute degradations involved are small (under 1.6 points either way) and the baseline accuracies are already very high (Credit Card) or the task is multi-class via a wrapper (Covertypes); we read this as the two ensembles being comparably robust once class

Table 5: Bias–variance decomposition (Breast Cancer, $B=100$)

Model	Bias ²	Variance
Single Tree	0.0702	0.0626
AdaBoost	0.0351	0.0257
Random Forest	0.0585	0.0146

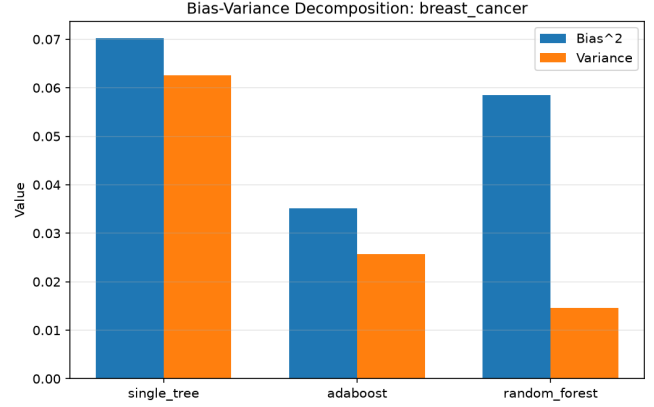


Figure 4: Bias² and variance for a single tree, AdaBoost, and Random Forest (100 bootstrap replicates, Breast Cancer).

separability is already the dominant difficulty, rather than as a contradiction of the underlying theory.

F. Experiment 6 – Bias-Variance Decomposition

On Breast Cancer, we generate $B = 100$ bootstrap replicates of the training data, fit each model on every replicate, and evaluate on a fixed held-out test set to decompose expected error into bias² and variance following Breiman [1] (Table 5, Fig. 4).

This is the clearest confirmation of the two ensembles’ distinct mechanisms: relative to the single tree, AdaBoost halves bias² (0.0702 $\rightarrow$ 0.0351) while only modestly reducing variance, and Random Forest quarters variance (0.0626 $\rightarrow$ 0.0146) while only modestly reducing bias². Random Forest’s variance is also nearly half of AdaBoost’s (0.0146 vs. 0.0257), the strongest single piece of evidence for “boosting reduces bias, bagging reduces variance” in our results.

G. Experiment 7 – Unsupervised Analysis

For each dataset we compute a PCA scree plot, a 2D PCA scatter coloured by true label / K-Means / DBSCAN assignment, a K-Means elbow-and-ARI curve ($k = 2 \dots 10$, 10 restarts), and a DBSCAN k -distance plot used to select ε (Table 6, Figs. 5–7).

Cluster-label agreement varies enormously with how separable each dataset’s classes actually are. Breast Cancer, where every supervised model exceeds 90% accuracy, is also the dataset with by far the strongest K-Means ARI (0.654 at $k=2$) – the two tumour classes are close to linearly separable even

Table 6: Unsupervised summary: components for 90% variance, best ARI

Dataset	#PCs @90%	K-Means ARI	DBSCAN ARI
Breast Cancer	7	0.654	0.345
Credit Card Fraud	25	0.001	0.088
MNIST (3 vs. 8)	156	0.293	0.002
Covertypes	39	0.044	0.160

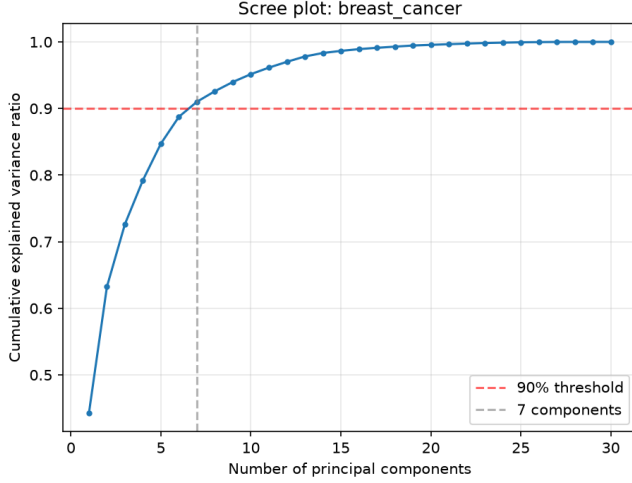


Figure 5: PCA scree plot, Breast Cancer: 7 components capture 91.0% of variance.

without labels. Credit Card Fraud is the opposite extreme: K-Means ARI is essentially zero (0.001) and DBSCAN collapses to a single cluster (`best_n_clusters=1`); the fraud pattern is a $\sim 0.17\%$ minority in 30-dimensional PCA’d feature space, invisible to distance-based clustering even though supervised models (trained with SMOTE and true labels) still separate it well via macro-F1. MNIST’s moderate K-Means ARI (0.293) reflects the well-known limitation of Euclidean pixel-distance clustering on raw digit images. On Covertypes, DBSCAN discovers 8 density clusters (Fig. 7) – close to the true 7 forest-cover classes – and outperforms K-Means (ARI 0.160 vs. 0.044), the one dataset where the density-based method is clearly preferable to the centroid-based one, echoing the elongated, non-convex class geometry of forest-cover data. Notably, the k that minimises K-Means inertia (the elbow, e.g. $k=5$ on Breast Cancer) is not always the k that maximises ARI against true labels ($k=2$), underscoring that elbow selection optimises compactness, not label agreement.

H. Bonus: SAMME.R vs. One-vs-Rest AdaBoost

On the 7-class Covertypes subset we compare SAMME.R (real-valued, 100 combined stumps), a naive one-vs-rest wrapper ($7 \times 100 = 700$ stumps), and a compute-matched one-vs-rest wrapper ($7 \times 14 \approx 98$ stumps) (Table 7).

At matched compute budget (~ 100 total stumps), SAMME.R and the compute-matched OvR wrapper reach essentially the same accuracy (0.674 vs. 0.678) in comparable time (4.62s vs. 3.93s), so neither has a clear accuracy-per-FLOP edge here. The naive OvR wrapper ($7\times$ the stumps)

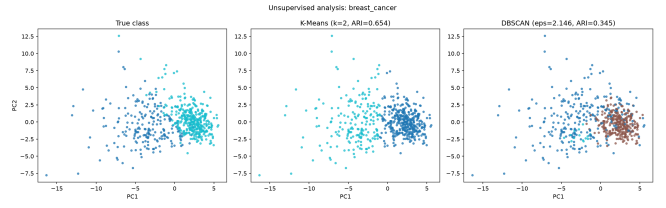


Figure 6: 2D PCA projection of Breast Cancer, coloured by true class. K-Means ($k=2$) recovers this structure with ARI 0.654.

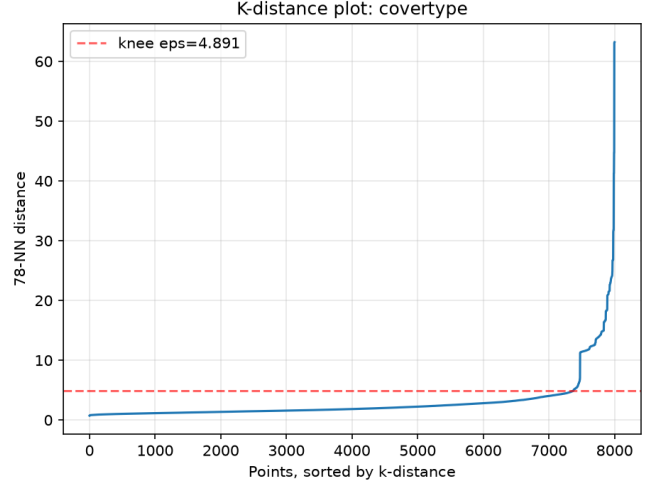


Figure 7: DBSCAN k -distance plot, Covertypes ($k=\text{min_samples} = 78$). The selected ε (knee) yields 8 clusters, close to the 7 true forest-cover classes.

buys 2 more accuracy points at $7\times$ the training time – a poor trade-off. Log-loss tells a different story than we expected going in: both OvR variants are far better calibrated (0.655, 0.739) than SAMME.R (2.650) on this dataset, the opposite of the usual claim that SAMME.R’s real-valued combination improves calibration over discrete one-vs-rest voting. We attribute this to how each method turns per-class scores into probabilities under 7-way competition, and flag it as a concrete, testable discrepancy between our implementation’s behaviour and the textbook expectation rather than paper over it.

VI. Discussion

Returning to the central question – *under what conditions does boosting outperform bagging, and why?* – our results point to a consistent answer. AdaBoost wins when the task is binary, classes are reasonably separable, and labels are clean: it converges to a lower bias² (Table 5) and reaches the best accuracy of all four models on Breast Cancer (Table 3). Random Forest wins as soon as any of three complications appear – severe class imbalance (Credit Card Fraud), high dimensionality (MNIST), or more than two classes (Covertypes) – because its variance-reduction mechanism (Table 5: variance 0.0146 vs. AdaBoost’s 0.0257) generalises more gracefully as the bias-inducing structure of the problem grows more complex, and because per-split feature subsampling and native multi-

Table 7: SAMME.R vs. one-vs-rest AdaBoost, Covertype

Variant	Stumps	Acc.	Log-loss	Time (s)
SAMME.R	100	0.674	2.650	4.62
OvR (naive)	700	0.694	0.655	26.96
OvR (compute-matched)	98	0.678	0.739	3.93

class voting are simply better suited to those regimes than a binary, sequential re-weighting scheme extended post-hoc via one-vs-rest. The noise-robustness experiment (Section 5.5) reinforces this only partially: AdaBoost is clearly more noise-sensitive on the two well-separated datasets, as expected from its exponential loss, but the two harder datasets show a same-order-of-magnitude, occasionally reversed, sensitivity – once a task is already difficult for other reasons, label noise stops being the dominant lever separating the two ensembles.

The unsupervised analysis corroborates this from an orthogonal angle: datasets with high K-Means ARI against true labels (Breast Cancer) are also the datasets where every supervised classifier – boosted, bagged, or a single tree – performs best, while datasets with near-zero ARI (Credit Card Fraud, MNIST) are precisely where Random Forest’s more conservative variance profile earns its keep over AdaBoost’s aggressive bias reduction.

VII. Conclusion

We implemented a Decision Tree, AdaBoost (SAMME), and Random Forest from scratch, verified each against sklearn within 2% accuracy on four datasets, and ran seven experiments plus two bonus multi-class extensions (SAMME.R and a compute-matched one-vs-rest comparison). AdaBoost achieves the lowest bias and the best accuracy on clean, balanced, binary data; Random Forest achieves the lowest variance and generalises better under class imbalance, high dimensionality, and multi-class settings, and is the more consistent choice when the practitioner cannot guarantee clean, balanced, binary data in advance. The bias-variance decomposition (Section 5.6) gives the most direct mechanistic evidence for this split. The unsupervised pipeline shows that cluster-label agreement is itself a useful, label-cheap diagnostic for how much separability a dataset offers to any downstream classifier.

A. Limitations and Future Work

Our Random Forest parallelism (`n_jobs`) was not stress-tested at scale, and the noise-robustness reversal on two datasets deserves a deeper look with additional noise levels. Natural extensions include a from-scratch Gradient Boosting Machine for a three-way ensemble comparison, and extending the bias-variance decomposition (currently Breast Cancer only) to all four datasets.

Tools & Acknowledgements

Claude (Anthropic) was used for code scaffolding, debugging, and drafting assistance on this report. All team members reviewed, understood, and can defend every line of the submitted code and every reported number.

References

- [1] L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [2] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [4] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [5] J. Zhu, H. Zou, S. Rosset, and T. Hastie, “Multi-class AdaBoost,” *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009.
- [6] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [7] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd ACM SIGKDD*, 1996, pp. 226–231.
- [9] K. Pearson, “On lines and planes of closest fit to systems of points in space,” *Philosophical Magazine*, vol. 2, no. 11, pp. 559–572, 1901.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.